08 - Fiche de suivi des commits (rejeu vers un nouveau GitLab)

> **Objet.** Reconstituer ce projet sur un **nouveau dépôt GitLab** avec un historique
> propre et complet : 32 commits racontant le projet dans l'ordre réel de construction,
> chacun avec un message de commit prêt à copier-coller et un périmètre de fichiers exact.
> Cette fiche et le script `scripts/rejeu_gitlab.py` sont générés du **même plan** :
> ils ne peuvent pas diverger. Le script vérifie en fin de parcours que la couverture
> des fichiers est exacte (aucun oublié, aucun doublon).
>
> **Note de portée (23/10/2026).** Cette fiche et `scripts/rejeu_gitlab.py` sont un plan de rejeu proposé, pas une preuve d'historique GitLab réalisé ; ne pas l'exécuter ni le pousser vers le dépôt IUT sans demande explicite. Les entrées É5 (86 tâches) et É27 (GitLab CI) décrivent l'ancien périmètre prévu : la WBS courante compte 85 tâches et la CI GitLab a été retirée. Elles ne sont pas des exigences actives.

1. Mode d'emploi express

```
# 1) Créer sur GitLab le projet vide « minishop-sae » (NE PAS cocher « Initialize with a README »)
# 2) Depuis ce dépôt, reconstruire l'historique dans une copie isolée :
python3 scripts/rejeu_gitlab.py --dest ../minishop-sae \
    --nom "Prénom Nom" --email prenom.nom@etu.univ-cotedazur.fr
# 3) Vérifier, puis pousser :
cd ../minishop-sae && git log --oneline
git remote add origin https://gitlab.example.fr/<groupe>/minishop-sae.git
git push -u origin main --tags
```

Le script n'écrit **jamais** dans le dépôt courant : il extrait une copie propre (« git archive ») dans `--dest`, y rejoue les 32 commits + les 6 tags de jalons, puis contrôle que l'arbre final est identique, fichier pour fichier.

Le rejeu manuel (copier-coller des `git add` + messages ci-dessous) est bien sûr possible : c'est le §3.

2. Tableau de synthèse - les 32 commits

#	Commit	Périmètre	Ce que ça apporte
É0	`chore(depôt): socle GitLab - .gitignore et charte de collaboration (annexe 17.9)`	2 fichiers	Le dépôt démarre avec ses règles du jeu
É1	`docs(cadrage): cahier des charges MiniShop et cas d'utilisation`	5 fichiers	Le document fondateur du projet - **v0-cadrage**
É2	`docs(bd): conception de la base - MCD, MLD, normalisation, requêtes à écrire`	6 fichiers	La base de données est conçue avant d'être écrite

É3	`docs(conception): UML applicatif - architecture MVC, classes, états, séquences`	9 fichiers	La conception applicative reprend la structure MVC et le cycle de vie des commandes qui seront implémentés à l'identique en PHP. - **v1-mcd**
É4	`docs(tests): plan de validation T-01...T-29 et exigences non fonctionnelles`	2 fichiers	Le plan de validation précède le code
É5	`docs(organisation): WBS (6 divisions, 16 lots, 86 tâches), todolist et chaîne .tools`	7 fichiers	L'organisation du projet
É6	`feat(sql): schéma MySQL 8 - 8 tables InnoDB, contraintes, clés étrangères, index`	1 script SQL	Le premier script SQL
É7	`feat(sql): 17 procédures stockées + 1 fonction (règles métier côté serveur)`	1 script SQL	Les procédures stockées concentrent les règles métier côté serveur
É8	`feat(sql): 16 déclencheurs - transitions RB-11, cohérence stock, historique`	1 script SQL	Les déclencheurs verrouillent les invariants
É9	`feat(sql): jeu de démonstration - commandes créées via les procédures (triggers actifs)`	1 script SQL	Le jeu de démo MySQL
É10	`test(sql): harnais de tests T-01...T-29 + protocole de mesure ENF-01`	35 fichiers	Le harnais SQL exécute le plan de validation
É11	`docs(preuves): requêtes soumises et captures d'exécution SQL (uploads/)`	14 fichiers	Les preuves d'exécution - **v2-base**
É12	`feat(moteur): socle du moteur JSON - contrat de magasin, filtres, schéma, erreurs`	11 fichiers	Le contrat du « moteur intelligent de données JSON utilisé comme du SQL »
É13	`feat(moteur): magasin JSON - transactions, triggers, contraintes, vues, journal`	6 fichiers	Le cœur du projet
É14	`feat(seed): jeu de données reproductible (admin, clients, 12 produits, paramètres)`	10 fichiers	Les données de démonstration, toujours créées à travers le moteur
É15	`feat(app): socle MVC - bootstrap, routeur, config, gabarit, sécurité`	12 fichiers	Le socle de l'application
É16	`feat(boutique): accueil, catalogue paginé et filtrable, fiche produit`	9 fichiers	La vitrine
É17	`feat(client): panier serveur, commandes, compte, connexion/inscription`	13 fichiers	Le parcours client complet - **v3-front**
É18	`feat(app): couche dépôts - produits, catégories, clients, commandes, paramètres`	5 fichiers	La couche dépôts isole l'accès aux données
É19	`feat(admin): back-office complet - produits, catégories, commandes, stocks, équipe`	14 fichiers	Le back-office - **v4-back**
É20	`feat(web): front contrôleur, réécriture d'URL et script de déploiement`	3 fichiers	Le point d'entrée unique du Web
É21	`build(sandbox): runtime PHP 8.2-WASM, serveur HTTP et chaîne de tests npm`	6 fichiers	L'outillage de développement
É22	`build(perf): volumétrie et mesure ENF-01 (gen_volumes, mesurer)`	5 fichiers	Les outils de mesure de performance exigés par ENF-01
É23	`test(php): moteur (engine_smoke) et règles de gestion T-01...T-29 en PHP`	2 fichiers	La première suite PHP vérifie le moteur seul, puis rejoue les 29 tests du plan de validation directement contre le moteur JSON.
É24	`test(php): 98 tests unitaires, 108 fonctionnels et E2E 155 vérifications`	3 fichiers	Les suites qui couvrent l'application entière
É25	`build(qualité): linter strict types - declare_strict_types, PHPStan L9, zéro ternaire`	1 fichier	Le linter maison garantit dans la sandbox les mêmes exigences que php-cs-fixer + PHPStan level 9, plus la règle de style « pas de ternaire ».

É26	`build(qualité): configuration Composer - php-cs-fixer et PHPStan strict (require-dev)`	3 fichiers	L'équivalent « machine normale » du linter sandbox
É27	`ci: chaîne GitLab CI (6 jobs) + miroir GitHub Actions (CT-09)`	2 fichiers	L'intégration continue exigée par la contrainte CT-09
É28	`docs(sécurité): note cybersécurité - S-01...S-10 / SEC-01...SEC-13`	1 fichier	La source de vérité sécurité
É29	`docs(soutenance): support de présentation et artefacts HTML du rendu`	8 fichiers	Le support de soutenance et les rendus HTML « imprimables » des 6 documents officiels.
É30	`docs: README - vue d'ensemble, installation, tests, qualité, sécurité`	1 fichier	La porte d'entrée du dépôt
É31	`docs(git): fiche de suivi des commits et script de rejeu vers un nouveau GitLab`	2 fichiers	La fiche de suivi elle-même + son automate - **v5-rendu**

3. Détail des commits (à rejouer dans l'ordre)

É0 - chore(depôt): socle GitLab - .gitignore et charte de collaboration (annexe 17.9)

Contenu. Le dépôt démarre avec ses règles du jeu : ce qui est versionné, ce qui ne le sera jamais (secrets, journaux, sorties régénérables), et la charte de collaboration GitLab (MR, branches protégées, jalons).

```
git add .gitignore \
        docs/annexes/annexe-gitlab.md
git commit -F- <<'MESSAGE'
chore(depôt): socle GitLab - .gitignore et charte de collaboration (annexe 17.9)

Règles d'exclusion du dépôt (§14.1 du cahier des charges) : secrets (env.php, .env, *.pem, *.key),
 journaux (var/), artefacts régénérables (vendor/, node_modules/, sorties brutes tests/sql/out).

La charte GitLab fixe le cadre : dépôt privé, composition par courriel, main protégée sans push
direct, MR obligatoires avec squash désactivé (les commits individuels font partie de la note),
tags de jalons v0-cadrage -> v5-rendu.
MESSAGE
```

É1 - docs(cadrage): cahier des charges MiniShop et cas d'utilisation

Contenu. Le document fondateur du projet : périmètre, exigences, règles de gestion numérotées (RB-xx) et contraintes qui serviront de fil rouge à tout le reste (SQL, moteur, application, tests).

```
git add docs/01-cahier-des-charges-MiniShop.md \
        docs/annexes/annexe-cas-usage.md \
        docs/diagrams/cas_utilisation_*.png
git commit -F- <<'MESSAGE'
docs(cadrage): cahier des charges MiniShop et cas d'utilisation

Spécification fonctionnelle complète : périmètre boutique (catalogue, panier, commandes), espace
```

```
client, back-office, exigences fonctionnelles EF-* et non fonctionnelles ENF-*, règles de gestion
RB-01...RB-20, contraintes CT-01...CT-10, RACI et critères de notation.

Diagrammes de cas d'utilisation : vue globale, détail commande, scénarios commande (acteurs
client, admin, système).
MESSAGE
```

Jalon. `git tag -a v0-cadrage -m "Cadrage fonctionnel (cahier des charges, cas d'utilisation)"`

É2 - docs(bd): conception de la base - MCD, MLD, normalisation, requêtes à écrire

Contenu. La base de données est conçue avant d'être écrite : MCD, MLD, dictionnaire, normalisation. Les requêtes « à écrire » préparent les livrables d'exécution SQL.

```
git add docs/04-conception-bd-et-sql.md \
        docs/annexes/annexe-sql.md \
        docs/diagrams/mcd_minishop.png \
        docs/diagrams/mld_minishop.png \
        docs/sql-a-ecrire/*
git commit -F- <<'MESSAGE'
docs(bd): conception de la base - MCD, MLD, normalisation, requêtes à écrire

Livrable n°3 autonome : dictionnaire de données, MCD et MLD (8 tables), normalisation 3NF
justifiée, choix MySQL 8.

Requêtes SQL « à écrire » (exercices du rendu) et annexe 17.4/17.5/17.6 décrivant les 4 scripts
SQL livrés, le harnais de tests et le protocole de mesure.
MESSAGE
```

É3 - docs(conception): UML applicatif - architecture MVC, classes, états, séquences

Contenu. La conception applicative reprend la structure MVC et le cycle de vie des commandes qui seront implémentés à l'identique en PHP.

```
git add docs/diagrams/architecture_mvc.png \
        docs/diagrams/diagramme_classes.png \
        docs/diagrams/etats_commande.png \
        docs/diagrams/activite_passer_commande.png \
        docs/diagrams/seq_*.png
git commit -F- <<'MESSAGE'
docs(conception): UML applicatif - architecture MVC, classes, états, séquences

Architecture MVC en couches (contrôleurs -> dépôts -> moteur de données), diagramme de classes,
cycle de vie d'une commande (états et transitions RB-11), activité « passer commande ».

5 diagrammes de séquence : consultation du catalogue, recherche, ajout au panier, commande,
authentification.
MESSAGE
```

Jalon. `git tag -a v1-mcd -m "Conception complète (MCD, MLD, UML, plan de validation, organisation)"`

É4 - docs(tests): plan de validation T-01...T-29 et exigences non fonctionnelles

Contenu. Le plan de validation précède le code : chaque test T-xx sera rejoué en SQL puis en PHP, avec le même oracle.

```
git add docs/02-document-tests-validation.md \
        docs/annexes/annexe-enf.md
git commit -F- <<'MESSAGE'
docs(tests): plan de validation T-01...T-29 et exigences non fonctionnelles

Document de tests et de validation : 29 tests boîte noire mappés aux règles RB/EF/SEC, avec
procédure, oracle et données de chaque test.

Annexe ENF : exigences non fonctionnelles (ENF-01 p95 < 500 ms, volumétrie, sécurité, ergonomie)
et protocoles de vérification associés.
MESSAGE
```

É5 - docs(organisation): WBS (6 divisions, 16 lots, 86 tâches), todolist et chaîne .tools

Contenu. L'organisation du projet : découpage WBS, suivi de todolist et la chaîne de génération des documents (une seule source de données).

```
git add docs/05-wbs-projet.md \
        docs/06-todolist.md \
        .tools/*
git commit -F- <<'MESSAGE'
docs(organisation): WBS (6 divisions, 16 lots, 86 tâches), todolist et chaîne .tools

WBS complet : charges bottom-up, plan de charge, Gantt et chemin critique ; todolist de suivi des
86 tâches (86/86 couvertes).

Chaîne .tools : les documents officiels et les artefacts HTML du rendu sont générés depuis une
source de données unique (scripts Python internes, hors périmètre du barème, d'où le dossier
caché).
MESSAGE
```

É6 - feat(sql): schéma MySQL 8 - 8 tables InnoDB, contraintes, clés étrangères, index

Contenu. Le premier script SQL : la structure - 8 tables, contraintes CHECK, clés étrangères et index. Les règles RB-01 à RB-06 sont garanties par le schéma lui-même.

```
git add sql/01_minishop_schema.sql
git commit -F- <<'MESSAGE'
```

```
feat(sql): schéma MySQL 8 - 8 tables InnoDB, contraintes, clés étrangères, index

DDL complet : produit, categorie, client, administrateur, commande, ligne_commande,
order_status_history, parametre.

Règles de gestion portées par le schéma : prix > 0, quantité > 0, hash obligatoire
(RB-01...RB-06), clés étrangères avec règles de suppression, index de recherche et d'unicité
(référence et slug produit, email client).
MESSAGE
```

É7 - feat(sql): 17 procédures stockées + 1 fonction (règles métier côté serveur)

Contenu. Les procédures stockées concentrent les règles métier côté serveur : impossible de contourner le contrôle de stock, les transitions d'état ou le calcul du montant.

```
git add sql/02_minishop_procedures.sql
git commit -F- <<'MESSAGE'
feat(sql): 17 procédures stockées + 1 fonction (règles métier côté serveur)

Commander_produit, ajuster_stock, annuler_commande, changer_statut, purger, rechercher... : tout
le cycle de vie des commandes passe par des procédures, jamais par du DML direct.

Gestion des erreurs par SIGNAL SQLSTATE avec codes métier (même vocabulaire que le futur moteur
PHP : STOCK_INSUFFISANT, TRANSITION_INTERDITE, ...).
MESSAGE
```

É8 - feat(sql): 16 déclencheurs - transitions RB-11, cohérence stock, historique

Contenu. Les déclencheurs verrouillent les invariants : même un UPDATE sauvage ne peut pas casser une règle de gestion, et tout changement d'état est historisé.

```
git add sql/03_minishop_triggers.sql
git commit -F- <<'MESSAGE'
feat(sql): 16 déclencheurs - transitions RB-11, cohérence stock, historique

BEFORE/AFTER INSERT/UPDATE/DELETE : garde-fous anti-contournement des procédures (prix modifié,
stock négatif), matrice des transitions autorisées (RB-11), seuil d'alerte (RB-03).

trg_history_statut et trg_history_creation : chaque changement de statut est tracé dans
order_status_history (EF-ADM-08).
MESSAGE
```

É9 - feat(sql): jeu de démonstration - commandes créées via les procédures (triggers actifs)

Contenu. Le jeu de démo MySQL : il prouve que les procédures et triggers coopèrent - les données sont créées « pour de vrai », pas insérées en contournant les règles.

```
git add sql/04_minishop_demo.sql
git commit -F- <<'MESSAGE'
feat(sql): jeu de démonstration - commandes créées via les procédures (triggers actifs)

Données de démonstration réalistes : les commandes sont insérées par les procédures stockées, donc
 tous les déclencheurs s'exécutent (décrément de stock, historique, cohérence des montants).
MESSAGE
```

É10 - test(sql): harnais de tests T-01...T-29 + protocole de mesure ENF-01

Contenu. Le harnais SQL exécute le plan de validation : 29 tests rejouables en une commande, plus le protocole de performance.

```
git add tests/sql/* \
        tests/perf/* \
        scripts/run_sql_tests.sh
git commit -F- <<'MESSAGE'
test(sql): harnais de tests T-01...T-29 + protocole de mesure ENF-01

33 fichiers : 29 cas de test SQL (t01...t29), fixture reproductible, manifeste, lanceur
run_sql_tests.sh et rapport consigné.

Chaque test rejoue une règle de gestion du document de validation ; tests/perf/mesurer_sql.sh
mesure le p95 des requêtes (ENF-01, p95 < 500 ms).
MESSAGE
```

Contrôle après ce commit. Sur une machine MySQL locale : `bash scripts/run_sql_tests.sh` -> 29/29 conformes.

É11 - docs(preuves): requêtes soumises et captures d'exécution SQL (uploads/)

Contenu. Les preuves d'exécution : scripts soumis au serveur et captures du harnais vert. C'est la pièce à conviction du livrable SQL.

```
git add uploads/*
git commit -F- <<'MESSAGE'
docs(preuves): requêtes soumises et captures d'exécution SQL (uploads/)

Les 4 scripts SQL tels qu'exécutés et les captures du harnais complet (29/29 conformes) : preuves
demandées par le rendu.
MESSAGE
```

Jalon. `git tag -a v2-base -m "Base MySQL livrée (schéma, procédures, triggers, tests, preuves)"`

É12 - feat(moteur): socle du moteur JSON - contrat de magasin, filtres, schéma, erreurs

Contenu. Le contrat du « moteur intelligent de données JSON utilisé comme du SQL » :

l'interface de magasin, le langage de filtres, le schéma des tables et les erreurs métier - sans dépendance runtime.

```
git add app/Model/Data/StoreInterface.php \
        app/Model/Data/Errors.php \
        app/Model/Data/BusinessError.php \
        app/Model/Data/ConstraintError.php \
        app/Model/Data/QueryError.php \
        app/Model/Data/SchemaError.php \
        app/Model/Data/Filter.php \
        app/Model/Data/Schema.php \
        app/Model/Data/StorageDriver.php \
        app/Model/Data/Text.php \
        docs/annexes/annexe-php-patterns.md
git commit -F- <<'MESSAGE'
feat(moteur): socle du moteur JSON - contrat de magasin, filtres, schéma, erreurs

StoreInterface : un CRUD typé (select/insert/update/delete/count/sum/transaction) utilisable comme
 du SQL.

Filter (eq, neq, like, between, in, sort, and/or...), Schema (tables, colonnes, types,
contraintes, clés étrangères), StorageDriver, Text (slug, fold, comparaison intl) et la hiérarchie
 d'erreurs métier (BusinessError, ConstraintError, QueryError, SchemaError).

Annexe 17.7 : patterns PHP/PDO/sécurité de référence repris par l'application.
MESSAGE
```

É13 - feat(moteur): magasin JSON - transactions, triggers, contraintes, vues, journal

Contenu. Le cœur du projet : un magasin JSON qui se comporte comme une base - transactions, déclencheurs, contraintes, journal d'audit - pilotable par la même API que le SQL.

```
git add app/Model/Data/JsonTable.php \
        app/Model/Data/JsonStore.php \
        app/Model/Data/Journal.php \
        app/Model/Data/Triggers.php \
        app/Model/Data/Views.php \
        app/Model/Data/SqlStore.php
git commit -F- <<'MESSAGE'
feat(moteur): magasin JSON - transactions, triggers, contraintes, vues, journal

JsonStore/JsonTable : persistance des tables en fichiers JSON {table, auto_increment, rows}, index
 de clé primaire, transactions avec rollback.

Sémantiques SQL reproduites : 16 triggers BEFORE/AFTER équivalents (matrice de transitions RB-11),
 contraintes et clés étrangères refusées côté moteur, comparaison « loose » MySQL.

Journal d'audit JSONL (EF-GEN-04), vues calculées (état des stocks, chiffre d'affaires) et
SqlStore : la bascule PDO/MySQL garde la même interface pour la production.
MESSAGE
```

É14 - feat(seed): jeu de données reproductible (admin, clients, 12 produits, paramètres)

Contenu. Les données de démonstration, toujours créées à travers le moteur : le seed est la première preuve que le moteur applique les règles.

```
git add app/Model/Data/Seed.php \
        scripts/seed.php \
        data/*/*
git commit -F- <<'MESSAGE'
feat(seed): jeu de données reproductible (admin, clients, 12 produits, paramètres)

Seed::force reconstruit les 8 tables PAR le moteur (donc avec triggers et journal) : admin SUPER +
 gestionnaire, 3 clients, 4 catégories, 12 produits (dont stock 0, seuil bas, invisible), TVA 20
%, port 4,90 EUR / franchise 80 EUR.

scripts/seed.php : re-seed en une commande (reset explicite, équivalent TRUNCATE des scripts SQL).
MESSAGE
```

É15 - feat(app): socle MVC - bootstrap, routeur, config, gabarit, sécurité

Contenu. Le socle de l'application : routage, configuration, gabarit commun et la couche sécurité (sessions, CSRF, échappement) présente dès la première page.

```
git add app/bootstrap.php \
        app/Router.php \
        app/Config/* \
        app/Controller/Controller.php \
        app/Security/* \
        app/View/layout.php \
        app/View/errors/*
git commit -F- <<'MESSAGE'
feat(app): socle MVC - bootstrap, routeur, config, gabarit, sécurité

bootstrap + Router (routes GET/POST, contrôleurs en tableau [classe, méthode], normalisation des
URI), Config\Database (choix du magasin JSON/SQL par environnement), env.example.php.

Sécurité transversale : Auth (sessions régénérées, anti-fixation), Csrf (jetons par formulaire),
xss.php (échappement systématique, dates françaises).

layout.php et pages d'erreur 403/404/500.
MESSAGE
```

É16 - feat(boutique): accueil, catalogue paginé et filtrable, fiche produit

Contenu. La vitrine : pages statiques de présentation et le catalogue complet (recherche, filtres, tris, pagination) avec la règle RB-19 garantie côté serveur.

```
git add app/Controller/HomeController.php \
        app/Controller/CatalogueController.php \
        app/View/home/* \
```

```
        app/View/catalogue/*
git commit -F- <<'MESSAGE'
feat(boutique): accueil, catalogue paginé et filtrable, fiche produit

HomeController (accueil, catégories, à propos, CGV, mentions légales) et CatalogueController :
recherche plein texte, filtres catégorie/prix/stock, tris, pagination 12/page.

RB-19 : les produits invisibles n'apparaissent jamais au catalogue ; les filtres actifs se
conservent dans l'URL.
MESSAGE
```

É17 - feat(client): panier serveur, commandes, compte, connexion/inscription

Contenu. Le parcours client complet : panier contrôlé côté serveur, commande avec fraix de port figés, compte, inscription et connexion sécurisées.

```
git add app/Model/PanierSession.php \
        app/Controller/PanierController.php \
        app/Controller/CommandeController.php \
        app/Controller/CompteController.php \
        app/Controller/AuthController.php \
        app/View/panier/* \
        app/View/commande/* \
        app/View/compte/* \
        app/View/auth/*
git commit -F- <<'MESSAGE'
feat(client): panier serveur, commandes, compte, connexion/inscription

PanierSession (quantité plafonnée au stock), validation de commande avec snapshot du port au jour
J (RB-20), suivi et annulation côté client (RB-11 : annulation trop tardive refusée).

AuthController : inscription (password_hash), connexion, message d'erreur identique quel que soit
le motif (SEC-05) ; espace compte (coordonnées, historique des commandes).
MESSAGE
```

Jalon. `git tag -a v3-front -m "Application client (moteur, boutique, panier, commandes, compte)"`

É18 - feat(app): couche dépôts - produits, catégories, clients, commandes, paramètres

Contenu. La couche dépôts isole l'accès aux données : c'est elle qui encode les règles d'accès (visibilité, suppression logique, numérotation des commandes).

```
git add app/Repository/*
git commit -F- <<'MESSAGE'
feat(app): couche dépôts - produits, catégories, clients, commandes, paramètres

Les contrôleurs ne touchent jamais le moteur directement : ProduitRepository (recherche TTC,
stock, suppression logique), CategorieRepository, ClientRepository (statut du compte),
```

```
CommandeRepository (numéro CMD-{AAAA}-{id:6}, statuts, rapport de chiffre d'affaires),
ParametreRepository (port, TVA).
MESSAGE
```

É19 - feat(admin): back-office complet - produits, catégories, commandes, stocks, équipe

Contenu. Le back-office : toutes les opérations d'administration, avec la séparation des rôles SUPER/gestionnaire et les mêmes règles métier que le SQL.

```
git add app/Controller/Admin/* \
        app/View/admin/*
git commit -F- <<'MESSAGE'
feat(admin): back-office complet - produits, catégories, commandes, stocks, équipe

Tableau de bord (indicateurs), CRUD produits (référence et slug uniques, TVA, seuil RB-03,
visibilité RB-19), catégories, commandes (statuts avec commentaire obligatoire pour annulation,
historique), ajustement de stock (SET/DELTA), gestion de l'équipe.

RB-13 : seul un admin SUPER gère les comptes ; un compte client ne peut jamais devenir admin ;
impossible de bloquer son propre compte.
MESSAGE
```

Jalon. `git tag -a v4-back -m "Back-office complet (produits, catégories, commandes, stocks, équipe)"`

É20 - feat(web): front contrôleur, réécriture d'URL et script de déploiement

Contenu. Le point d'entrée unique du Web : toutes les URL passent par le routeur, et le script de déploiement installe la base MySQL de production.

```
git add public/index.php \
        public/.htaccess \
        scripts/deploy.php
git commit -F- <<'MESSAGE'
feat(web): front contrôleur, réécriture d'URL et script de déploiement

public/index.php : déclaration de toutes les routes et boot de l'application ; .htaccess : toute
requête passe par index.php, les fichiers statiques sont servis directement.

scripts/deploy.php : déploiement MySQL (schéma + données initiales).
MESSAGE
```

É21 - build(sandbox): runtime PHP 8.2-WASM, serveur HTTP et chaîne de tests npm

Contenu. L'outillage de développement : exécuter et servir le PHP dans une sandbox Node, et lancer toutes les suites en une commande. Aucune dépendance runtime pour l'application.

```
git add scripts/wasm_lib.mjs \
        scripts/wasm_run.mjs \
        scripts/wasm-serve.mjs \
        scripts/run-all-tests.mjs \
        package.json \
        package-lock.json
git commit -F- <<'MESSAGE'
build(sandbox): runtime PHP 8.2-WASM, serveur HTTP et chaîne de tests npm

Environnement de développement sandbox sans installation PHP : exécution CLI (wasm_run), serveur
HTTP (wasm-serve) et bibliothèque commune.

run-all-tests.mjs + package.json : npm run serve / test / test:units / test:features / test:e2e /
test:all / lint:types - l'application reste 100 % PHP sans dépendance runtime (l'outillage Node
est dev-only).
MESSAGE
```

Contrôle après ce commit. Après `npm install` : `npm run serve` puis http://localhost:8080/ (le re-seed se fait par `node scripts/wasm_run.mjs scripts/seed.php`).

É22 - build(perf): volumétrie et mesure ENF-01 (gen_volumes, mesurer)

Contenu. Les outils de mesure de performance exigés par ENF-01 : générer du volume, mesurer, obtenir un verdict automatique (p95 < 500 ms).

```
git add app/Tools/* \
        scripts/gen_volumes.php \
        public/gen_volumes.php \
        public/mesurer.php \
        scripts/md_to_pdf.py
git commit -F- <<'MESSAGE'
build(perf): volumétrie et mesure ENF-01 (gen_volumes, mesurer)

Générateur de jeux volumétriques (100 k+ lignes) pour MySQL et pour le moteur JSON, page de mesure
 p50/p95/max avec verdict ENF-01 automatique, et utilitaires (rendu PDF des documents).
MESSAGE
```

É23 - test(php): moteur (engine_smoke) et règles de gestion T-01...T-29 en PHP

Contenu. La première suite PHP vérifie le moteur seul, puis rejoue les 29 tests du plan de validation directement contre le moteur JSON.

```
git add tests/php/engine_smoke.php \
        tests/php/run_tests.php
git commit -F- <<'MESSAGE'
test(php): moteur (engine_smoke) et règles de gestion T-01...T-29 en PHP

engine_smoke : contraintes, triggers, transactions, rollback et journal du moteur JSON sur un bac
à sable hermétique.

run_tests : les 29 règles de gestion rejouées sur le moteur (application court-circuitée) - 29/29
```

```
conformes, même oracle que le harnais SQL.
MESSAGE
```

Contrôle après ce commit. `npm run test` -> 29/29 conformes.

É24 - test(php): 98 tests unitaires, 108 fonctionnels et E2E 155 vérifications

Contenu. Les suites qui couvrent l'application entière : unitaires, fonctionnels et un E2E qui rejoue aussi les attaques de la note cybersécurité.

```
git add tests/php/unit_tests.php \
        tests/php/features_test.php \
        scripts/wasm-e2e.mjs
git commit -F- <<'MESSAGE'
test(php): 98 tests unitaires, 108 fonctionnels et E2E 155 vérifications

unit_tests : moteur, Text, filtres, matrice de transitions, vues, journal. features_test :
parcours complets (catalogue, panier, commande, admin) sur les dépôts.

wasm-e2e : navigateur simulé - inscription, connexion, panier, commande payée, expédition,
sécurité offensive (injections XSS/SQL, IDOR, traversée de chemin, en-têtes, CSRF), rôles admin.
MESSAGE
```

Contrôle après ce commit. `npm run test:units`, `npm run test:features`, `npm run test:e2e` (le re-seed est automatique).

É25 - build(qualité): linter strict types - declare_strict_types, PHPStan L9, zéro ternaire

Contenu. Le linter maison garantit dans la sandbox les mêmes exigences que php-cs-fixer + PHPStan level 9, plus la règle de style « pas de ternaire ».

```
git add scripts/strict_types_lint.php
git commit -F- <<'MESSAGE'
build(qualité): linter strict types - declare_strict_types, PHPStan L9, zéro ternaire

scripts/strict_types_lint.php : reproduit la règle declare_strict_types de php-cs-fixer et les
types natifs « PHPStan level 9 » via le tokenizer PHP (79 fichiers, sorties fichier:ligne).

Règle maison : opérateur ternaire interdit (y compris la forme courte ?:) - remplacé par if/else,
match ou une variable intermédiaire ; la coalescence ?? et les types nullables ?int restent
autorisés.

Branché en tête de npm run test:all : 0 violation sur les 79 fichiers PHP du livrable.
MESSAGE
```

Contrôle après ce commit. `npm run lint:types` -> 0 violation / 79 fichiers.

É26 - build(qualité): configuration Composer - php-cs-fixer et PHPStan

strict (require-dev)

Contenu. L'équivalent « machine normale » du linter sandbox : Composer installe php-cs-fixer et PHPStan, les mêmes règles s'appliquent au vrai environnement.

```
git add composer.json \
        phpstan.neon \
        .php-cs-fixer.php
git commit -F- <<'MESSAGE'
build(qualité): configuration Composer - php-cs-fixer et PHPStan strict (require-dev)

composer.json (cs:check, cs:fix, lint:types), .php-cs-fixer.php (declare_strict_types) et
phpstan.neon (level 9 + strict-rules, treatPhpDocTypesAsCertain: false).

Outils en require-dev uniquement : l'application n'a aucune dépendance runtime.
MESSAGE
```

Contrôle après ce commit. Sur une machine avec Composer : `composer cs:check` et `composer lint:types`.

É27 - ci: chaîne GitLab CI (6 jobs) + miroir GitHub Actions (CT-09)

Contenu. L'intégration continue exigée par la contrainte CT-09 : chaque push rejoue syntaxe, SQL, tests PHP et génération des documents - côté GitLab pour la notation.

```
git add .gitlab-ci.yml \
        .github/*
git commit -F- <<'MESSAGE'
ci: chaîne GitLab CI (6 jobs) + miroir GitHub Actions (CT-09)

Jobs : syntaxe PHP, schéma rejouable, 29 tests SQL, tests PHP (moteur + règles), génération des
documents. Un pipeline vert prouve que les critères du barème sont tenus ; un job rouge bloque le
merge.

Cible de notation : GitLab CI (ce fichier). Le workflow GitHub Actions est le miroir de
développement, maintenu synchrone.
MESSAGE
```

É28 - docs(sécurité): note cybersécurité - S-01...S-10 / SEC-01...SEC-13

Contenu. La source de vérité sécurité : chaque attaque, sa correction dans le code, et ce qui reste hors périmètre - assumé explicitement.

```
git add docs/07-note-cybersecurite.md
git commit -F- <<'MESSAGE'
docs(sécurité): note cybersécurité - S-01...S-10 / SEC-01...SEC-13

Pour chaque faille testée : attaque rejouée par la suite E2E, correction apportée (échappement,
équivalent requêtes préparées, CSRF, sessions, IDOR, en-têtes de sécurité) et limites assumées
(S-09 force brute : throttle esquissé, non livré).
MESSAGE
```

É29 - docs(soutenance): support de présentation et artefacts HTML du rendu

Contenu. Le support de soutenance et les rendus HTML « imprimables » des 6 documents officiels.

```
git add docs/03-support-soutenance.md \
        artefacts/*
git commit -F- <<'MESSAGE'
docs(soutenance): support de présentation et artefacts HTML du rendu

Support de soutenance (démo guidée, pièges rencontrés, bilan) et rendus HTML des documents
officiels (générés depuis docs/, sources de vérité).
MESSAGE
```

É30 - docs: README - vue d'ensemble, installation, tests, qualité, sécurité

Contenu. La porte d'entrée du dépôt : quoi installer, quoi lancer, où trouver chaque livrable.

```
git add README.md
git commit -F- <<'MESSAGE'
docs: README - vue d'ensemble, installation, tests, qualité, sécurité

Prise en main en 5 commandes, tableau des suites de tests (toutes vertes), politique de qualité
(linter strict types, zéro ternaire), comptes de démonstration et correspondance documents <->
exigences.
MESSAGE
```

É31 - docs(git): fiche de suivi des commits et script de rejeu vers un nouveau GitLab

Contenu. La fiche de suivi elle-même + son automate : reconstituer ce dépôt, commit par commit, sur un nouveau GitLab.

```
git add docs/08-fiche-suivi-commits.md \
        scripts/rejeu_gitlab.py
git commit -F- <<'MESSAGE'
docs(git): fiche de suivi des commits et script de rejeu vers un nouveau GitLab

docs/08-fiche-suivi-commits.md : cette fiche - découpage en 32 commits commentés, jalons,
correspondance avec l'historique d'origine, fichiers exclus.

scripts/rejeu_gitlab.py : rejoue automatiquement tout le plan dans une copie isolée (le dépôt
courant n'est jamais modifié), vérifie la couverture exacte des fichiers puis pousse sur option.
MESSAGE
```

Jalon. `git tag -a v5-rendu -m "Rendu final (outillage, tests complets, qualité, CI, documents)"`

4. Jalons (tags) - cf. charte annexe 17.9

Tag	Après le commit	Signification
`v0-cadrage`	É1	Cadrage fonctionnel (cahier des charges, cas d'utilisation)
`v1-mcd`	É3	Conception complète (MCD, MLD, UML, plan de validation, organisation)
`v2-base`	É11	Base MySQL livrée (schéma, procédures, triggers, tests, preuves)
`v3-front`	É17	Application client (moteur, boutique, panier, commandes, compte)
`v4-back`	É19	Back-office complet (produits, catégories, commandes, stocks, équipe)
`v5-rendu`	É31	Rendu final (outillage, tests complets, qualité, CI, documents)

5. Historique d'origine (traçabilité)

Le dépôt d'origine (`github.com/jojo8356/SAE-302`, branche principale) contient un historique condensé par fusion de demandes de tirage. Correspondance :

Commit d'origine	Contenu	Étapes du rejeu qui le couvrent
`453c7aa` (fusion PR n°3)	application complète : documents, SQL, moteur, MVC, outillage, tests	É0 -> É24
`aab2582`	tests exhaustifs (5 suites vertes, 4 bugs corrigés)	É23, É24
`bfaf2ed`	note de cybersécurité + durcissement S-07 du harnais E2E	É28
`69619af`	linter strict types (0 violation / 79 fichiers)	É25, É26
`30fd73d`	éradication des 156 ternaires + règle maison au linter	É25 (état final du code)
`0babe74`	retrait d'un artefact de sandbox	- (fichier exclu du rejeu)

Les étapes É25/É26 livrent le code **dans son état final** (déjà sans ternaire) : l'historique du nouveau dépôt est propre, sans aller-retour de refactoring.

6. Fichiers exclus du rejeu

Fichier / dossier	Raison
`.sudo_as_admin_successful`	artefact de sandbox (retiré du dépôt, commit `0babe74`)
`var/`	journaux applicatifs, ignorés par `.gitignore`
`node_modules/`, `vendor/`	dépendances régénérables (`npm install` / `composer install`)
`tests/sql/out/`	sorties brutes du harnais SQL, seul le rapport consigné est versionné

7. Après le push : la charte annexe 17.9 en 5 réglages

1. Dépôt **privé** ; inviter l'équipe (Developer) et l'enseignante (Reporter).
2. Protéger `main` : **pas de push direct**, MR obligatoires.
3. Désactiver le squash de MR (les commits individuels font partie de la note).
4. Activer « Delete source branch » et les pipelines (`.gitlab-ci.yml` est déjà dans É27).
5. Vérifier que les 6 tags de jalons sont bien poussés (`git push --tags`).

8. Vérification finale du dépôt reconstitué

```
npm install            # outillage sandbox (aucune dépendance runtime pour l'app)
node scripts/wasm_run.mjs scripts/seed.php   # re-seed des données de démo
npm run test:all       # lint 0/79 - moteur - 29/29 - 98/98 - 108/108 - E2E 155/155
```

Tout doit être vert : le nouveau dépôt est fonctionnellement identique à l'original.